



ST. JOHN PAUL II COLLEGE OF DAVAO

College of Information and Communications Technology

CC104 - Data Structures and Algorithms

MINI-SYSTEM PROJECT

Implementation of a Priority Queue (Binary Max-Heap) in an Online Scholarship Application System

Researchers

[Full Names of Group Members]

Group Leader

[Full Name]

Course

Data Structures and Algorithms

Instructor

John Patrick P. Eleria

Section

[Section Name/Code]

Institution

St. John Paul II College of Davao

Date of Submission

[Month, Day, Year]

TABLE OF CONTENTS

Chapter I - Introduction.....	2
1.1 Background of the Study.....	2
1.2 Objectives of the Study.....	2
1.3 Significance of the Study.....	3
Chapter II - System Overview and Design.....	4
2.1 System Description.....	4
2.2 System Architecture and Flowchart.....	4
2.3 System Features.....	6

Chapters III to VI, References, and Appendices are not included in this submission. They are scheduled for the succeeding laboratory examinations.

CHAPTER I - INTRODUCTION

1.1 Background of the Study

Scholarship programs in higher education institutions routinely receive far more applications than there are slots to award. A scholarship office therefore does not need the applications kept in full sorted order at all times. What it needs, repeatedly and on demand, is the answer to a single question: among the applications still waiting, who is the most deserving right now? Applications keep arriving while that question is being asked, so any approach that re-sorts the entire list after every new submission spends most of its work ordering applicants who will never be reached.

This study addresses that mismatch by building an Online Scholarship Application System around a priority queue implemented as a binary max-heap. A binary max-heap is a complete binary tree stored in a plain array, where the element at index i keeps its parent at index $(i - 1) / 2$ and its children at indices $2i + 1$ and $2i + 2$. The structure maintains one invariant, the heap property: every parent holds a priority greater than or equal to both of its children. The tree is therefore only partially ordered, which is precisely why it is inexpensive to maintain, yet the single element the scholarship office actually needs, the highest priority applicant, is always at the root and can be read in constant time.

In this system the priority of an application is a merit score from 0 to 100 that the system computes from three weighted components: academic standing from the general weighted average, worth up to 60 points; financial need from the reported monthly household income, worth up to 30 points; and the enrolled unit load for the term, worth up to 10 points. Because two applicants can earn exactly the same score, the queue breaks ties in favour of whoever submitted first, which keeps the ordering total, deterministic, and fair. Submitting an application is an insertion that sifts the new applicant up toward the root, and awarding a scholarship slot is an extract-max that removes the root and sifts its replacement back down. Both operations touch at most one node per level of the tree, so each costs $O(\log n)$ rather than the $O(n \log n)$ a full re-sort would cost.

The prototype implements the heap from first principles in a class named `ApplicantPriorityQueue` rather than delegating to the built-in `java.util.PriorityQueue`, so the index arithmetic, the sift-up loop, and the sift-down loop are all visible and can be explained line by line. The heap is the true core of the system: no sorting routine decides who is awarded a slot, and the ordering is never recomputed from scratch. To make the structure observable rather than merely claimed, the browser interface renders the live backing array in level order beside the tree it represents, and reports the number of comparisons and swaps each operation performed. Applications are held in the memory of the running Java process and are cleared when that process restarts.

1.2 Objectives of the Study

The general objective of this study is to implement a binary max-heap priority queue as the core mechanism of an Online Scholarship Application System, and to observe how that structure maintains the ordering of applicants as applications arrive and scholarship slots are awarded.

Specifically, this study aims:

1. To implement the insert, extract-max, and peek operations of a binary max-heap from first principles over an array-backed complete binary tree, using only parent and child index arithmetic rather than stored node links.
2. To convert each submitted application into a single merit score from 0 to 100 derived from academic standing, financial need, and enrolled unit load, and to settle equal scores by order of submission so the resulting ordering is total and deterministic.
3. To enqueue every validated application through a sift-up so that the applicant holding the highest merit score always occupies the root of the heap.
4. To award each scholarship slot through a single extract-max with a sift-down, so the most deserving waiting applicant is released without re-sorting the remaining applications.
5. To expose the backing array, the tree it represents, the fully ranked waiting list, and the comparison and swap counts of each operation, so the behaviour of the heap can be inspected directly rather than inferred.
6. To validate every application in Java before it reaches the structure, so that a rejected submission leaves the queue and the awarded slots unchanged.

1.3 Significance of the Study

This study is significant to the following:

Researchers. The project moves the priority queue from a definition to be memorised into a structure the researchers had to build, debug, and defend. Writing the sift-up and sift-down loops by hand and proving through testing that the heap property survives thousands of mixed operations taught the trade-off at the centre of the structure: a heap gives up full ordering in exchange for cheap access to the single element that matters.

Data Structures and Algorithms students. The interface shows the backing array and the tree side by side, which makes visible the point that is hardest to convey from a textbook diagram alone. A student can see that the array is genuinely not sorted, that only the root is guaranteed to be the maximum, and that an insertion still finds its place after only a handful of comparisons.

Instructors. The prototype offers a working example in which a priority queue is the core mechanism of a realistic system rather than a decorative add-on. The reported comparison and swap counts give an immediate, concrete demonstration of logarithmic growth that can be shown during a lecture without instrumentation.

Scholarship coordinators. The system models the real constraint a scholarship committee works under: a fixed number of slots and a continuing stream of applicants. Because the merit score is broken down into its academic, financial-need, and unit-load components on screen, the ranking is explainable to an applicant who asks why another was selected first.

Scholarship applicants. An applicant can see the exact basis of the ranking, including the position held in the queue and the points earned in each component. The submission-order tie-break also guarantees that applying earlier is never a disadvantage against an identically qualified candidate.

CHAPTER II - SYSTEM OVERVIEW AND DESIGN

2.1 System Description

The Online Scholarship Application System is a single Spring Boot application that both serves the browser interface and exposes the JSON endpoints behind it. An applicant is entered through a web form that collects five values: the applicant's name, the degree program, the general weighted average, the reported monthly household income, and the number of units enrolled for the term. The browser performs required-field checks first, but Java holds the authoritative validation, so a request that bypasses the browser is still rejected and the queue is left untouched.

Once an application is accepted, the system assigns it a submission sequence number and a reference code of the form SCH-0001, then computes its merit score. The score is the sum of three clamped components: a general weighted average of 1.00 earns the full 60 academic points and 3.00 or worse earns none; a monthly household income of zero earns the full 30 need points and an income of 60,000 pesos or more earns none; and 24 units or more earns the full 10 load points. That single number becomes the applicant's priority inside the queue, with the submission sequence number acting as the tie-breaker when two applicants score identically.

The scored applicant is then inserted into an ApplicantPriorityQueue, the hand-written binary max-heap that is the core of the system. The applicant is appended at the first free position of the backing array, which keeps the binary tree complete, and is then swapped upward with its parent for as long as it outranks that parent. When the scholarship office awards a slot, the system performs one extract-max: it takes the root, moves the last element into the root position, and sinks that element by repeatedly swapping it with its stronger child until the heap property holds again. The remaining applications are never re-sorted, and the number of scholarship slots is a configurable limit that defaults to five, after which further award requests are refused.

The interface presents the same data in three deliberately different views. The raw backing array is shown in level order with each element's index and parent index, making clear that the array is only partially ordered. A tree diagram renders the same array as the complete binary tree it represents, with the root highlighted. A separate ranked table lists every waiting applicant from highest to lowest merit, produced by running a heap sort over a copy of the array so that reading the ranking never disturbs the live queue. Each operation also reports the comparisons and swaps it performed. All applications are held in the memory of the running Java process, so the queue and the awarded slots reset whenever that process restarts.

2.2 System Architecture and Flowchart

The architecture keeps the browser responsible only for collecting input and presenting results, while Java owns validation, merit scoring, the heap itself, and the decision of who is awarded next. No ranking logic exists in JavaScript. Figure 1 shows the implemented flow, including the recoverable error path that retains the values the applicant typed and the branch taken when a scholarship slot is awarded.

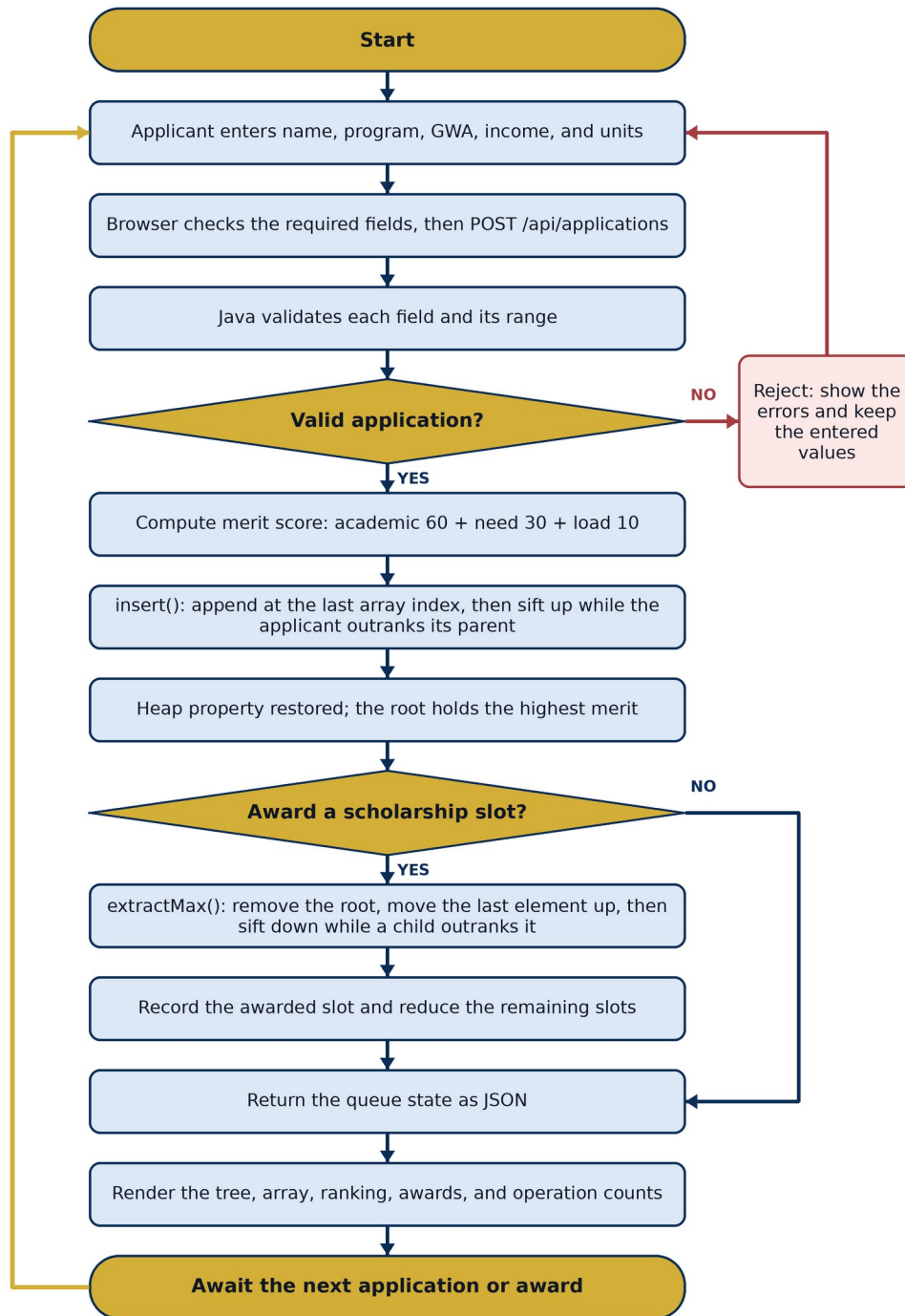


Figure 1. Online Scholarship Application System Flowchart

Input. The applicant's name, degree program, general weighted average, monthly household income, and units enrolled, entered through the browser form. The general weighted average must fall from 1.00 through 5.00, the income from 0 through 1,000,000 pesos, and the unit load from 1 through 36. A second form of input is the award request raised by the scholarship office when a slot is to be granted.

Process. The browser checks the required fields, then posts the application to Java. Java validates the fields and their ranges, assigns the submission sequence and reference code, and computes the merit score from its academic, financial-need, and unit-load components. The scored applicant is appended to the backing array and sifted up until its parent outranks it, restoring the heap property. When a slot is awarded instead, the system removes the root, moves the last element into its place, and sifts that element down until the heap property holds again, then records the awarded slot.

Output. The service returns the complete queue state as JSON: the backing array in level order, the fully ranked waiting list, the applicant currently at the root, the awarded slots, the remaining slot count, the height and capacity of the heap, and a re-check of the heap property. JavaScript renders the tree, the array table, the ranked table, the awarded table, and the comparison and swap counts of the operation just performed, then leaves the interface ready for the next action.

2.3 System Features

1. Accepts a scholarship application consisting of an applicant name, degree program, general weighted average, monthly household income, and units enrolled, through a responsive browser form.
2. Validates every field in Java, rejecting blank names, out-of-range averages, negative or excessive incomes, and invalid unit loads, while leaving the queue and the awarded slots unchanged and retaining the values the applicant typed.
3. Computes a merit score from 0 to 100 for each application, combining up to 60 points for academic standing, up to 30 points for financial need, and up to 10 points for the enrolled unit load, and displays the three components beside the total.
4. Assigns each accepted application a submission sequence number and a reference code of the form SCH-0001.
5. Stores every waiting application in a hand-written binary max-heap held in an array, where a parent at index i keeps its children at indices $2i + 1$ and $2i + 2$.
6. Inserts each accepted applicant with a sift-up so the highest merit applicant always occupies the root of the heap.
7. Breaks an exact tie in merit score in favour of the applicant who submitted first, making the ordering total and deterministic.
8. Awards a scholarship slot with a single extract-max and sift-down, releasing the highest merit waiting applicant without re-sorting the remaining applications.
9. Enforces a configurable limit on scholarship slots, defaulting to five, and refuses further award requests once every slot has been granted or when no applications are waiting.

10. Displays the live backing array in level order with each element's index and parent index, alongside a tree diagram of the same array with the root highlighted.
11. Produces a fully ranked list of waiting applicants by running a heap sort over a copy of the array, so reading the ranking never modifies the live queue.
12. Reports the number of comparisons and swaps performed by each insertion and extraction, together with a readable trace of every swap and the current height of the tree.
13. Re-checks the heap property across the whole array after every operation and reports the result in the interface.
14. Grows the backing array by doubling it when the tree fills it, so the queue is not limited by a capacity fixed in advance.
15. Loads a fixed set of sample applications on request, including a late-arriving strongest applicant and an exact tie, so the structure can be demonstrated without typing, and clears the queue and awarded slots on request.